

Assignment 01: 2Sum and Complexity

Talan Jewett | CS 210

1. Overview

This program solves the classic Two Sum problem: given an integer vector *nums* and an integer *target*, return the indices of two different elements whose values add to *target*. The assignment requires two implementations with the same signature `std::pair<int, int>` so their behavior and performance can be compared directly:

- **twoSumBruteForce** — nested loops, no hash table.
- **twoSumHash** — single pass with an `unordered_map` for expected linear time.

2. Design Decisions

Brute force. For every index *i*, the inner loop only checks *j > i* rather than re-checking pairs already covered by earlier *i* values. This avoids checking (*i*, *j*) and (*j*, *i*) as if they were distinct, and it naturally excludes pairing an element with itself. It returns as soon as the first valid pair is found.

Hash-based. While scanning left to right, the function computes *complement* = *target* - *nums*[*i*] and checks whether that value has already been recorded in the map *before* inserting the current value. Checking first and inserting second is what guarantees the two indices returned are always different — if the array contained *target*/2 twice, checking-then-inserting still requires the complement to have come from an earlier, distinct index.

Multiple valid pairs. The assignment states that exactly one valid *pair* can be assumed to exist, but the required test array actually contains more than one pair of values that sum to 24. Because the two algorithms visit elements in different orders (brute force compares farthest-apart-first within a fixed *i*; the hash version stops at the first complement it has already seen), they can legitimately return two different index pairs for the same input, as shown in Section 4. Both are correct: the problem only requires a valid pair, not a unique one. The test harness checks correctness (do the indexed values sum to the target, and are the indices distinct and in range?) rather than checking for one specific expected pair, so this ambiguity does not cause false failures.

3. Complete Source Code

File: two_sum.cpp (C++17)

```
// Assignment 01: 2Sum and Complexity
// CS 210
//
// Given an integer vector `nums` and an integer `target`, return the
// indices of two different elements whose values add to `target`.
// Exactly one valid pair is assumed to exist.

#include <iostream>
#include <vector>
#include <unordered_map>
#include <utility>
#include <string>

// -----
// Approach 1: Brute force (nested loops, no hash table).
// For every pair (i, j) with i < j, check whether nums[i] + nums[j]
// equals target. Stops at the first match, which is fine here because
// the assignment guarantees exactly one valid pair exists.
// -----
std::pair<int, int> twoSumBruteForce(const std::vector<int>& nums, int target) {
    int n = static_cast<int>(nums.size());
    for (int i = 0; i < n; ++i) {
        for (int j = i + 1; j < n; ++j) {
            if (nums[i] + nums[j] == target) {
                return {i, j};
            }
        }
    }
    // No valid pair found (shouldn't happen given the assignment's guarantee).
    return {-1, -1};
}

// -----
// Approach 2: Hash-based lookup, single pass.
// While scanning left to right, we ask "have I already seen the value
// that completes this pair?" before inserting the current value. That
// ordering also naturally prevents using the same element twice.
// Expected O(1) per lookup/insert with unordered_map, so overall
// expected O(n) time.
// -----
std::pair<int, int> twoSumHash(const std::vector<int>& nums, int target) {
    std::unordered_map<int, int> seen; // value -> index
    int n = static_cast<int>(nums.size());
    for (int i = 0; i < n; ++i) {
        int complement = target - nums[i];
        auto it = seen.find(complement);
        if (it != seen.end()) {
            return {it->second, i};
        }
        seen[nums[i]] = i;
    }
    // No valid pair found (shouldn't happen given the assignment's guarantee).
    return {-1, -1};
}

// -----
// Test harness
// -----
void runCase(const std::string& label, const std::vector<int>& nums, int target) {
    std::cout << "=== " << label << " ===\n";
    std::cout << "target = " << target << "\n";
```

```

std::pair<int, int> bf = twoSumBruteForce(nums, target);
std::pair<int, int> hs = twoSumHash(nums, target);

auto report = [&](const std::string& name, std::pair<int, int> result) {
    bool inRange = result.first >= 0 && result.second >= 0 &&
        result.first < static_cast<int>(nums.size()) &&
        result.second < static_cast<int>(nums.size());
    bool distinct = result.first != result.second;
    int sum = inRange ? nums[result.first] + nums[result.second] : 0;
    bool valid = inRange && distinct && sum == target;

    std::cout << " " << name << ": indices = (" << result.first << ", "
        << result.second << ")";
    if (inRange) {
        std::cout << ", values = (" << nums[result.first] << ", "
            << nums[result.second] << "), sum = " << sum;
    }
    std::cout << ", valid = " << (valid ? "true" : "false") << "\n";
};

report("twoSumBruteForce", bf);
report("twoSumHash", hs);
std::cout << "\n";
}

int main() {
    // Required test case from the assignment.
    runCase("Required case",
        {15, 4, 18, 8, 19, 22, 24, 59, 59, 20, 18, 12, 36, 42, 9}, 24);

    // Additional case 1: negative numbers, target 0.
    runCase("Negative numbers, target 0", {-3, 4, 3, 90}, 0);

    // Additional case 2: the matching pair is a repeated value.
    runCase("Duplicate values as the pair", {3, 3, 7, 11}, 6);

    // Additional case 3: the matching pair sits at the very end.
    runCase("Pair at the end of the array", {1, 2, 3, 4, 5}, 9);

    // Additional case 4: negative target.
    runCase("Negative target", {5, -2, 10, -7, 1}, -9);

    return 0;
}

```

4. Test Cases and Output Evidence

Compiled with: `g++ -std=c++17 -Wall -Wextra -O2 -o two_sum two_sum.cpp` — no warnings or errors. The program below runs the required test case plus four additional cases chosen to cover negative numbers, a duplicate value serving as the matching pair, a solution at the very end of the array, and a negative target.

```
=== Required case ===
target = 24
    twoSumBruteForce: indices = (0, 14), values = (15, 9), sum = 24, valid = true
    twoSumHash       : indices = (1, 9), values = (4, 20), sum = 24, valid = true

=== Negative numbers, target 0 ===
target = 0
    twoSumBruteForce: indices = (0, 2), values = (-3, 3), sum = 0, valid = true
    twoSumHash       : indices = (0, 2), values = (-3, 3), sum = 0, valid = true

=== Duplicate values as the pair ===
target = 6
    twoSumBruteForce: indices = (0, 1), values = (3, 3), sum = 6, valid = true
    twoSumHash       : indices = (0, 1), values = (3, 3), sum = 6, valid = true

=== Pair at the end of the array ===
target = 9
    twoSumBruteForce: indices = (3, 4), values = (4, 5), sum = 9, valid = true
    twoSumHash       : indices = (3, 4), values = (4, 5), sum = 9, valid = true

=== Negative target ===
target = -9
    twoSumBruteForce: indices = (1, 3), values = (-2, -7), sum = -9, valid = true
    twoSumHash       : indices = (1, 3), values = (-2, -7), sum = -9, valid = true
```

5. Time and Space Complexity

twoSumBruteForce

Time: $O(n^2)$. The outer loop runs n times, and for each i the inner loop runs up to $n - i - 1$ times, giving $n(n-1)/2$ comparisons in the worst case (the matching pair is the last two elements, or no pair exists). Best case is $O(1)$ if the very first pair checked happens to match, but that is not the case the complexity of an algorithm is judged by.

Space: $O(1)$. Only a constant number of loop counters are used; no auxiliary data structure grows with the input.

twoSumHash

Time: $O(n)$ expected. The array is scanned once. Each iteration performs one `find` and one `insert` on the `unordered_map`, each $O(1)$ on average (amortized, assuming a reasonable hash distribution). n iterations $\times O(1)$ work = $O(n)$ expected time. (Worst case, with many hash collisions, degrades toward $O(n^2)$, but this is not the typical or expected case for integer keys.)

Space: $O(n)$. In the worst case — the matching complement is not found until near the end of the array — the map stores nearly every element seen so far, so it can grow to size proportional to n .

Why the hash-based approach improves on brute force

Brute force has no memory of the elements it has already examined, so for every new starting index it re-scans the remaining array from scratch, repeating work it has effectively already done. That repetition is exactly what produces the n^2 growth rate. The hash-based approach spends $O(n)$ extra memory to remember every value it has seen and its index, which turns each subsequent lookup into an $O(1)$ operation instead of an $O(n)$ scan. This is the classic time/space tradeoff: trading a linear amount of extra memory for a quadratic-to-linear improvement in running time.

Concretely, for the 15-element required test case, brute force performs up to $15 \times 14 / 2 = 105$ comparisons in the worst case, while the hash version performs at most 15 lookups and 15 inserts. The gap widens dramatically as n grows: at $n = 10,000$, brute force is on the order of 50 million comparisons versus roughly 10,000 hash operations for the hash-based version — the difference between a program that finishes instantly and one that visibly lags as input size increases.